MICROSAR vHsm_Hal

Technical Reference

vHsm Hardware Abstraction Layer

Version 3.03.00

Authors	Alexander Diegel, Matthias Hanreich, Jan Hieber, Frederik Peukert, Florian Reumann
Status	Released

Document Information

History

Author	Date	Version	Remarks
Jan Hieber	2019-03-14	2.02.00	Initial creation
Florian Reumann	2019-03-14	3.00.00	Added KeyManagement functions (3.5 and 5.1.3), usage of ROM-Keys (3.5.1)
Frederik Peukert Matthias Hanreich Alexander Diegel	2019-11-05 2019-11-15 2019-11-20	3.01.00	Added Flash Synchronization chapter (3.5.4.1). Added Custom definitions (3.5.3) Added Limitations for AEAD (3.1.3) Added CTR mode to 3.1 Added chapter TRNG prefetching (3.6) Added critical sections (4.2)
Hanreich Matthias	2020-04-20	3.03.00	Added API for checking for debugger presence (5.1.5.7)

Reference Documents

No.	Source	Title	Version
[1]	AUTOSAR	AUTOSAR_SWS_CryptoDriver.pdf	4.3
[2]	AUTOSAR	AUTOSAR_SWS_CryptoInterface.pdf	4.3
[3]	AUTOSAR	AUTOSAR_SRS_CryptoStack.pdf	4.3
[4]	AUTOSAR	AUTOSAR_SWS_DET.pdf	4.3
[5]	AUTOSAR	AUTOSAR_TR_BSWModuleList.pdf	4.3
[6]	Vector	vHsm_IntegrationHints_<Platform>.pdf	see delivery
[7]	NIST	NIST Special Publication 800-108	n.a.

Contents

1 Component History 7

2 Introduction 8
2.1 Architecture Overview 9

3 Functional Description 10
3.1 Features 10
3.1.1 Deviations 10
3.1.2 Additions/ Extensions 11
3.1.3 Limitations 11
3.1.3.1 AEAD Initialization Vector 11
3.1.3.2 AEAD Input Length 11
3.1.3.3 AEAD Det Checks 11
3.1.3.4 Providing of Additional Authenticated Data for AEAD 12
3.1.3.5 Providing output when tag verification fails 12
3.1.3.6 Length of output buffer for streaming use case for AEAD 12
3.1.3.7 Size of output buffer in finish step for streaming use case for AEAD and AES-128 CTR 13
3.2 Initialization 13
3.3 Main Functions 13
3.4 Error Handling 13
3.4.1 Development Error Reporting 13
3.5 Key Management 14
3.5.1 Usage of ROM-Keys 14
3.5.2 Key Derivation 15
3.5.3 Custom definitions 16
3.5.3.1 Custom Key Elements 16
3.5.3.2 Custom Key Element Values 17
3.5.4 Custom Services 17
3.5.4.1 Flash Access Synchronization 17
3.6 True Random Number Generator 19
3.6.1 Prefetching of randomness 19

4 Integration 20
4.1 Scope of Delivery 20
4.1.1 Static Files 20
4.1.2 Dynamic Files 21
4.2 Critical Sections 21
4.3 Configuration 21

4.4 Integration Hints 22

5 API Description 23
5.1 Services provided by vHsm_Hal 23
5.1.1 General services 23
5.1.1.1 vHsm_Hal_Init 23
5.1.1.2 vHsm_Hal_InitMemory 23
5.1.1.3 vHsm_Hal_MainFunction 24
5.1.2 Crypto services 25
5.1.2.1 vHsm_Hal_ProcessJob 25
5.1.2.2 vHsm_Hal_CancelJob 25
5.1.3 Key Management services 26
5.1.3.1 vHsm_Hal_KeyElementGet 26
5.1.3.2 vHsm_Hal_KeyElementSet 27
5.1.3.3 vHsm_Hal_KeyDerive 27
5.1.3.4 vHsm_Hal_KeyValidSet 28
5.1.4 Platform services 29
5.1.4.1 vHsm_Hal_GetDynamicMemoryAreaConfig 29
5.1.4.2 vHsm_Hal_SystemInit 29
5.1.4.3 vHsm_Hal_TriggerSoftwareReset 30
5.1.4.4 vHsm_Hal_HandleResetInfo 30
5.1.4.5 vHsm_Hal_StartupCallout 31
5.1.4.6 vHsm_Hal_TriggerInternalIrq 31
5.1.4.7 vHsm_Hal_ApplicationIpcInit 32
5.1.4.8 vHsm_Hal_EnableInterruptSource 32
5.1.4.9 vHsm_Hal_DisableInterruptSource 33
5.1.5 Optional callouts 33
5.1.5.1 vHsm_Hal_GetCodeFlashAddr 33
5.1.5.2 vHsm_Hal_PrepareCryptoJobData 34
5.1.5.3 vHsm_Hal_PrepareKeyMJobData 34
5.1.5.4 vHsm_Hal_GetIpcAddress 35
5.1.5.5 vHsm_Hal_EnableCodeFlashAccess 35
5.1.5.6 vHsm_Hal_DisableCodeFlashAccess 36
5.1.5.7 vHsm_Hal_IsDebuggerPresent 36
5.1.6 Interrupt Service Routines provided by VHSM_HAL 37
5.1.7 Services used by VHSM_HAL 37

6 Configuration 39
6.1 Configuration Variants 39

7 Glossary and Abbreviations 40

7.1 Glossary 40
7.2 Abbreviations 40

8 Contact 41

Illustrations

Figure 2-1 AUTOSAR 4.3 Architecture Overview 9
Figure 2-2 Interfaces to adjacent modules of the VHSM_HAL 9
Figure 3: Example key configuration 16

Tables

Table 1-1 Component history 7
Table 3-1 Supported AUTOSAR standard conform features 10
Table 3-2 Not supported AUTOSAR standard conform features 10
Table 3-3 Features provided beyond the AUTOSAR standard 11
Table 3-4 Overview of the required algorithm parameter for AEAD 12
Table 3-5 Service IDs 14
Table 3-6 Errors reported to DET 14
Table 3-7 Custom Key Element 17
Table 3-8 Custom Key Element Values 17
Table 4-1 Static files 21
Table 4-2 Generated files 21
Table 5-1 vHsm_Hal_Init 23
Table 5-2 vHsm_Hal_InitMemory 24
Table 5-3 vHsm_Hal_MainFunction 24
Table 5-4 vHsm_Hal_ProcessJob 25
Table 5-5 vHsm_Hal_CancelJob 26
Table 5-6 vHsm_Hal_KeyElementGet 26
Table 5-7 vHsm_Hal_KeyElementSet 27
Table 5-8 vHsm_Hal_KeyDerive 28
Table 5-9 vHsm_Hal_KeyValidSet 28
Table 5-10 vHsm_Hal_GetDynamicMemoryAreaConfig 29
Table 5-11 vHsm_Hal_SystemInit 29
Table 5-12 vHsm_Hal_TriggerSoftwareReset 30
Table 5-13 vHsm_Hal_HandleResetInfo 30
Table 5-14 vHsm_Hal_StartupCallout 31
Table 5-15 vHsm_Hal_TriggerInternalIrq 31
Table 5-16 vHsm_Hal_ApplicationIpcInit 32
Table 5-17 vHsm_Hal_EnableInterruptSource 32
Table 5-18 vHsm_Hal_DisableInterruptSource 33
Table 5-19 vHsm_Hal_GetCodeFlashAddr 34
Table 5-20 vHsm_Hal_PrepareCryptoJobData 34
Table 5-21 vHsm_Hal_PrepareKeyMJobData 35
Table 5-22 vHsm_Hal_GetIpcAddress 35
Table 5-23 vHsm_Hal_EnableCodeFlashAccess 36
Table 5-24 vHsm_Hal_DisableCodeFlashAccess 36
Table 5-25 vHsm_Hal_IsDebuggerPresent 37
Table 5-26 Services used by the VHSM_HAL 38
Table 7-1 Glossary 40
Table 7-2 Abbreviations 40

1 Component History

The component history gives an overview over the important milestones that are supported in the different versions of the component.

Component Version	New Features
2.00.00	Initial release.
2.00.01	Support native CMAC hardware accelerators.
2.01.00	Optimization of CMAC subkey calculation. Restructured block processing for AES and CMAC to enable the use of pipelining if supported by the hardware.
2.02.00	Support usage of SHE keys.
3.00.00	Support of ROM Keys, Key Derivation in Counter Mode with CMAC-128 as PRF
3.01.00	Support for AEAD and AES-CTR mode, TRNG prefetching, protection of HW access by critical sections, support for interrupts by application.
3.02.00	Performance optimization for job processing
3.03.00	Readout of debugger presence if supported by the hardware, Support of configurations with no vHSM_HAL crypto driver objects

Table 1-1 Component history

2 Introduction

This document describes the functionality, API and configuration of the AUTOSAR BSW module VHSM_HAL as specified in [1].

Supported AUTOSAR Release*:	4.3	
Supported Configuration Variants:	pre-compile	
Vendor ID:	VHSM_HAL_VENDOR_ID	30 decimal (= Vector-Informatik, according to HIS)
Module ID:	VHSM_HAL_MODULE_ID	114 decimal (according to ref. [5])

* For the detailed functional specification please also refer to the corresponding AUTOSAR SWS.

The vHsm_Hal is an AUTOSAR Crypto Driver as specified in [1]. It provides access to the crypto hardware accelerators offered by the platform. This means that there is a small subset of crypto primitives implemented by this module, but not all. There is also no ordinary key management available, although it can be implemented for special functions on some platforms.

The module also provides access to special features of the hardware by offering an API. This includes common functions like triggering a reset or interrupt and reading the cause of the last reset.

2.1 Architecture Overview

The following figure shows where the VHSM_HAL is located in the AUTOSAR architecture.

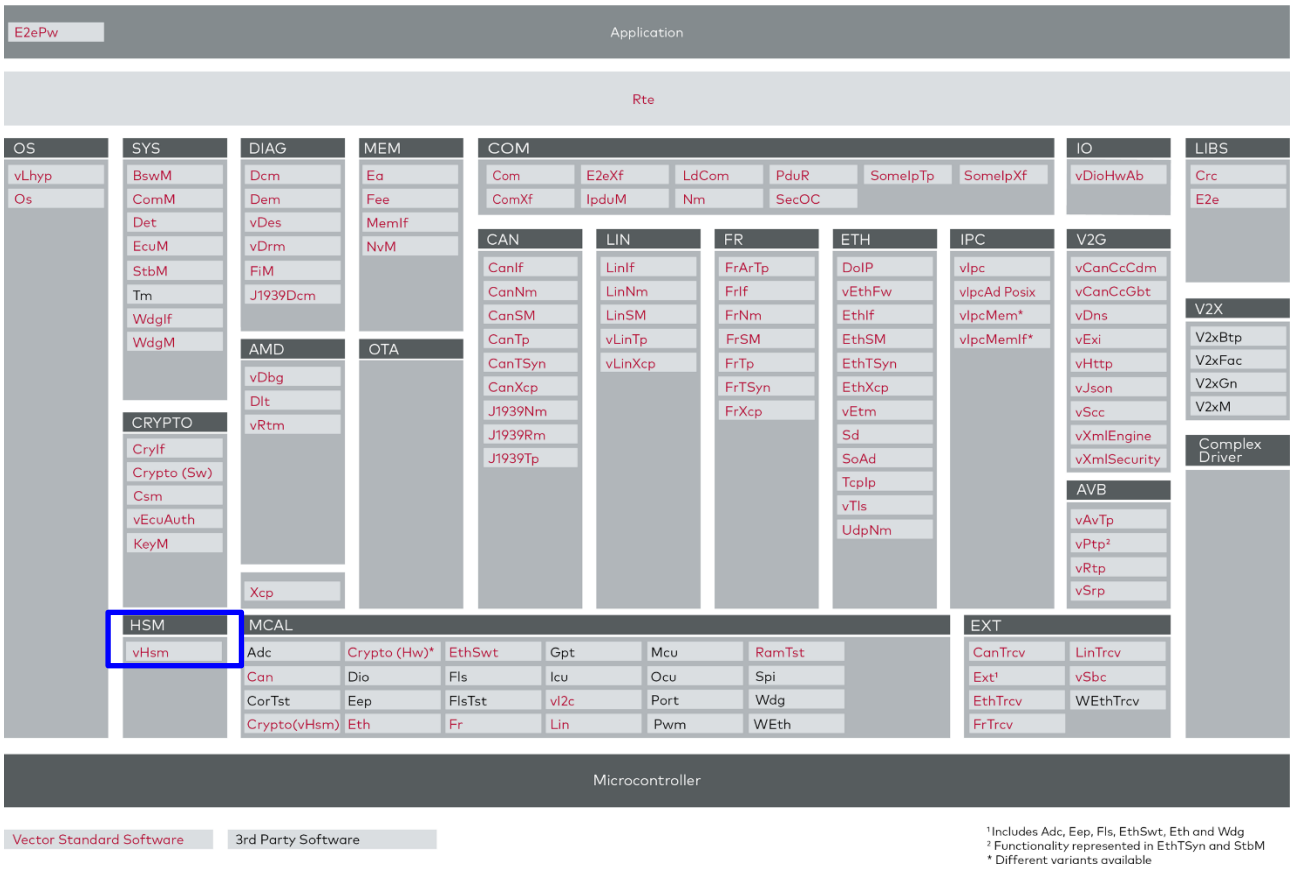


Figure 2-1 AUTOSAR 4.3 Architecture Overview

The next figure shows the interfaces to adjacent modules of the VHSM_HAL. These interfaces are described in chapter 5.

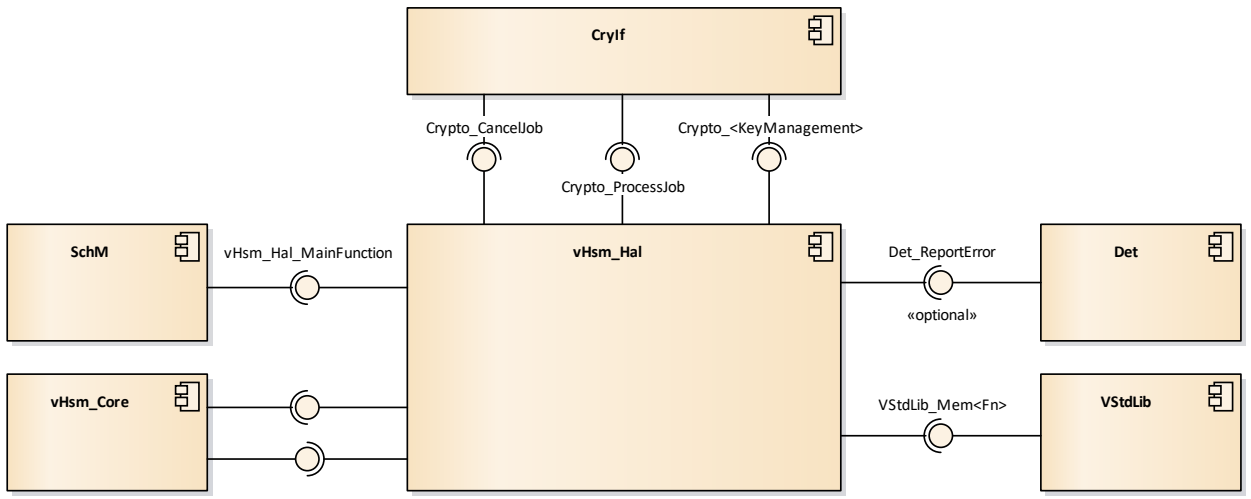


Figure 2-2 Interfaces to adjacent modules of the VHSM_HAL

3 Functional Description

3.1 Features

The features listed in the following tables cover the complete functionality specified for the VHSM_HAL.

The AUTOSAR standard functionality is specified in [1], the corresponding features are listed in the tables

> Table 3-1 Supported AUTOSAR standard conform features

> Table 3-2 Not supported AUTOSAR standard conform features

Vector Informatik provides further VHSM_HAL functionality beyond the AUTOSAR standard. The corresponding features are listed in the table

> Table 3-3 Features provided beyond the AUTOSAR standard

The following features specified in [1] are supported.

Supported AUTOSAR Standard Conform Features
AEAD: AES-128 GCM (only if supported by the platform)
AES-128 ECB/CBC with PKCS7 padding and without padding, CTR without padding
TRNG
MAC: AES CMAC
HASH: SHA-256 (only if supported by the platform)
Key Derivation: Counter Mode with CMAC-AES128 as PRF according to NIST SP800-108:2009

Table 3-1 Supported AUTOSAR standard conform features

3.1.1 Deviations

The following features specified in [1] are not supported:

Not Supported AUTOSAR Standard Conform Features
All algorithms not stated in Table 3-1
Only KeyElementSet, KeyElementGet and KeyDerive are implemented by this Crypto driver. All other Key management functions are forwarded to vHsm_Core module.
API function are prefixed with **vHsm_Hal_** instead of **Crypto_**. Compatibility defines are provided with the modules implementation.
There is no support for asynchronous jobs, all crypto jobs are handled synchronously.

Table 3-2 Not supported AUTOSAR standard conform features

3.1.2 Additions/ Extensions

The following features are provided beyond the AUTOSAR standard:

Features Provided Beyond The AUTOSAR Standard
Multiple CryptoDriverObjects to support multiple workspaces per service. This is dependent on the platform specific hardware accelerators.
Use vStdLib for memory access
Provide APIs for hardware functions like: vHsm_Hal_HandleResetInfo() vHsm_Hal_TriggerSoftwareReset()
Provide API for prefetching random numbers from the TRNG module.

Table 3-3 Features provided beyond the AUTOSAR standard

3.1.3 Limitations

3.1.3.1 AEAD Initialization Vector

The length of the Initialization Vector is fixed to 96 bits.

3.1.3.2 AEAD Input Length

The AEAD implementation uses AES-128-GCM as cryptographic algorithm. AES-128-GCM does not require the input length to be a multiple of the AES block length. However, some hardware platforms can only process input with the length being a multiple of the AES block length. See the technical documentation of the specific hardware platforms for further information.



Caution

Padding the input to a multiple of the AES block length affects the result of the cryptographic operation. The padding is treated by AES-128 GCM as part of the input Therefore, different tags are calculated for input with padding and input without padding.

The Crypto driver does not alter the input with padding. This would falsify the expected result by calculating a different tag and may make it not possible to further process the output with other cryptographic implementations.

Should a hardware platform require the input to be a multiple of the AES block length, the Crypto driver expects the received input to already fulfill this requirement. Else, the job is rejected.

3.1.3.3 AEAD Det Checks

The Det checks for AEAD differ from the table in [1]. There is a different handling required for the AEAD implementation, as described in Table 3-4:

Service \ Member	inputPtr	inputLength	secondaryInputPtr	secondaryInputLength	tertiaryInputPtr	tertiaryInputLength	outputPtr	outputLengthPtr	secondaryOutputPtr	secondaryOutputLengthPtr	verifyPtr	mode
AEADENCRYPT	U	U	V	U~			UF	UF	F	F		SUF
AEADDECRYPT	U	U	V	U~	F	F	UF	UF			F	SUF

S: member required in Start mode.
U: member required in Update mode.
V: member optional in Update mode.
F: member required in Finish mode.
~: no Det check required / 0 is valid value.

Table 3-4 Overview of the required algorithm parameter for AEAD

3.1.3.4 Providing of Additional Authenticated Data for AEAD

The implementation of AEAD supports streaming. However, the optional Additional Authenticated Data (AAD) must be provided completely during the first update. Subsequent updates to AAD are ignored and do not influence the calculation of the tag.

3.1.3.5 Providing output when tag verification fails

A general security feature of AEAD is that no information about the decrypted plaintext is returned in case the tag verification fails. This cannot be guaranteed for the AEAD implementation. As specified by AUTOSAR in [1], parts of the plaintext are already returned during update mode for the streaming approach, before the correctness of the tag was verified.



Note
As a job can always be executed with the streaming approach, preventing the plaintext from being returned for the single call approach does not improve the security. Therefore, the plaintext is also returned for the single call approach in case the tag verification fails.

3.1.3.6 Length of output buffer for streaming use case for AEAD

For the streaming use case, it is not trivial to predict how many bytes of the plaintext- and ciphertext are processed in a certain update step. It is therefore recommended to track the returned output length of all update steps. This number can be used to compare how many bytes have already been processed in contrast to the provided input data and to calculate the required size of the output buffer for following update steps and the finish step.

3.1.3.7 Size of output buffer in finish step for streaming use case for AEAD and AES-128 CTR

For AEAD and AES-128 CTR, the final block of the plaintext- and ciphertext is always processed and returned in the finish step. Therefore, the provided output buffer in the finish step needs to be at least the size of the remaining input.

3.2 Initialization

Before any other functionality of the CRYPTO module can be called the initialization function `vHsm_Hal_Init()` has to be called by the BSWM.

For the initialization of RAM variables, the CRYPTO offers the function `vHsm_Hal_InitMemory()` which must be called before `vHsm_Hal_Init()`.

3.3 Main Functions

The CRYPTO module implementation provides one main function. The function has to be called cyclically.

3.4 Error Handling

3.4.1 Development Error Reporting

By default, development errors are reported to the DET using the service `Det_ReportError()` as specified in [4] , if development error reporting is enabled (i.e. pre-compile parameter `VHSM_HAL_DEV_ERROR_DETECT==STD_ON`).

If another module is used for development error reporting, the function prototype for reporting the error can be configured by the integrator, but must have the same signature as the service `Det_ReportError()`.

The reported VHSM_HAL ID is 114.

The reported service IDs identify the services which are described in 5.1. The following table presents the service IDs and the related services:

Service ID	Service
0x00	vHsm_Hal_Init()
0x01	vHsm_Hal_GetVersionInfo()
0x03	vHsm_Hal_ProcessJob()
0x0E	vHsm_Hal_CancelJob()
0x04	vHsm_Hal_KeyElementSet()
0x05	vHsm_Hal_KeyValidSet()
0x06	vHsm_Hal_KeyElementGet()
0x0F	vHsm_Hal_KeyElementCopy()
0x10	vHsm_Hal_KeyCopy()

Service ID	Service
0x11	vHsm_Hal_KeyElementIdsGet()
0x0D	vHsm_Hal_RandomSeed()
0x07	vHsm_Hal_KeyGenerate()
0x08	vHsm_Hal_KeyDerive()
0x09	vHsm_Hal_KeyExchangeCalcPubVal()
0x0A	vHsm_Hal_KeyExchangeCalcSecret()
0x0B	vHsm_Hal_CertificateParse()
0x12	vHsm_Hal_CertificateVerify()
0x0C	vHsm_Hal_MainFunction()

Table 3-5 Service IDs

The errors reported to DET are described in the following table:

Error Code	Description
0x00	CRYPTO_E_UNINIT
0x01	CRYPTO_E_INIT_FAILED
0x02	CRYPTO_E_PARAM_POINTER
0x04	CRYPTO_E_PARAM_HANDLE
0x05	CRYPTO_E_PARAM_VALUE

Table 3-6 Errors reported to DET

3.5 Key Management

3.5.1 Usage of ROM-Keys

The CRYPTO module supports the usage of keys which are preconfigured by the hardware manufacturer. It depends on the hardware platform whether this feature is supported and how many ROM-Keys are provided. The value of the ROM-Keys can only be accessed inside the hardware and cannot be written or read out. To utilize a ROM-Key, an associated key has been preconfigured, which can be used for a cryptographic job. The CRYPTO module takes care of the hardware specific configurations.

For cryptographic operations which require additional key elements, e.g. an initialization vector, it is necessary to configure a key with the according key element and add a reference to the field “Key Aggregation Ref” of the ROM-Key. Both keys are then aggregated by the CRYPTO module and managed as one key. The value of the key element must be set via the preconfigured ROM-Key. Adding an aggregated key is optional and can be omitted for cryptographic operations which only require a key.

3.5.2 Key Derivation

The key derivation implements NIST800-108 in counter mode[7]. For the parameter description see chapter 5.1.3.3 and for the custom definitions see chapter 3.5.3.

The key derivation can be activated by adding the subcontainer /MICROSAR/vHsm_Hal/Crypto/CryptoGeneral/CryptoKeyDerive.

The behavior of the key derivation algorithm is configured by the following custom key elements.

These elements are (see also Table 3-7 Custom Key Element).
a password (CRYPTO_KE_KEYDERIVATION_PASSWORD),
a context (CRYPTO_KE_KEYDERIVATION_CONTEXT),
and the element label (CRYPTO_KE _KEYDERIVATION_LABEL).

These elements are used as input for the PRF (Pseudo Random Function). For the PRF the CMAC-Function is used. Therefore, the CMAC-Function must be activated for the selected driver object.

The input for the PRF is generated by concatenating the elements.



Basic Knowledge

Concatenation of the input:
counter [1-4] || **label** || **0x00** || **context** || **length** [1-4]

counter: Counter of iterations
length: Length of Target Key
label: The label key element from the source key. (can be empty).
context: The context key element from the source key. (can be empty).
[1-4]: Number of Bytes for this element (configurable in DaVinci).
0x00: Zero octet to separate label and context.



Note

The configurable parameter MaxInputLength must be bigger than all the elements for the input together.

See: /MICROSAR/vHsm_Hal/Crypto/CryptoGeneral/CryptoKeyDerive/CryptoMaxInputLength

The configurable parameter MaxOutputLength must be bigger than the largest key become derived.

See: /MICROSAR/vHsm_Hal/Crypto/CryptoGeneral/CryptoKeyDerive/CryptoMaxOutputKeyLength

If no ROM-Key is passed the key is read from the key element CRYPTO_KE_KEYDERIVATION_PASSWORD of the source key. The derived key is stored in the key element CRYPTO_KE_KEYDERIVATION_TARGET_KEY_ELEMENT of the target key. The length of the derived key is set by the size of this element.

In addition to the above key elements the key element CRYPTO_30_VHSM_ALGO_KDF_NIST_800_108_CMAC must be present with the source key.

If a ROM-Key is used, another key with the key element's CRYPTO_KE_KEYDERIVATION_LABEL, CRYPTO_KE_KEYDERIVATION_CONTEXT and CRYPTO_30_VHSM_ALGO_KDF_NIST_800_108_CMAC must be aggregated to the ROM-Key. If a key aggregated to a Rom-Key contains a CRYPTO_KE_KEYDERIVATION_PASSWORD key element, this is ignored and the Rom-Key is used for the key derivation.

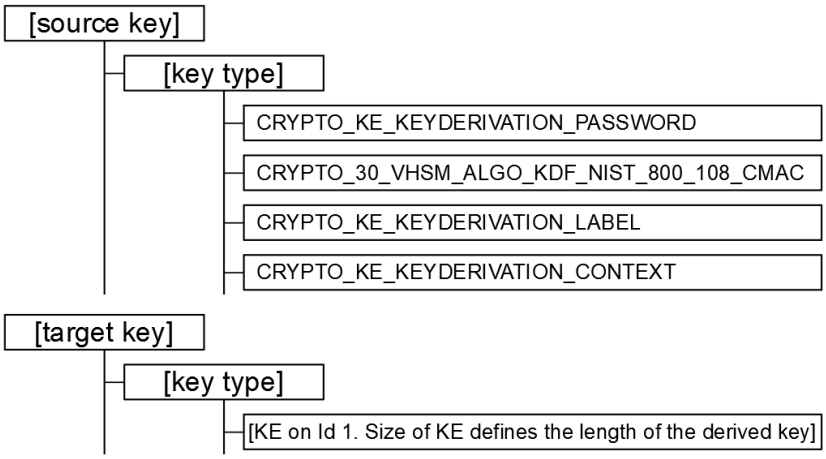


Figure 3: Example key configuration

Figure 3 shows one possible key configuration for the key derivation. The key elements for the source key contain the information for the key derivation. The target key contains the space for the derived key.

3.5.3 Custom definitions

3.5.3.1 Custom Key Elements

There are custom key elements defined, which are shown in the following table and are used for custom services described in chapter 3.5.4.

Custom Key Element	Id
CRYPTO_KE_CUSTOM_VHSM_FLASH_OPERATION	3032
CRYPTO_KE_CUSTOM_VHSM_KEYDERIVATION_ALGORITHM	15

CRYPTO_KE_KEYDERIVATION_PASSWORD	1
CRYPTO_KE_KEYDERIVATION_TARGET_KEY	1
CRYPTO_KE_KEYDERIVATION_CONTEXT	132
CRYPTO_KE_KEYDERIVATION_LABEL	130

Table 3-7 Custom Key Element

3.5.3.2 Custom Key Element Values

Some key elements only accept specific values, which are shown in the following table and are used for custom services described in chapter 3.5.4.

Key Element	Define	Value
CRYPTO_KE_CUSTOM_VHSM_FLASH_OPERATION	CRYPTO_30_VHSM_DATAFLASH_START	0
	CRYPTO_30_VHSM_DATAFLASH_STOP	1
	CRYPTO_30_VHSM_CODEFLASH_START	2
	CRYPTO_30_VHSM_CODEFLASH_STOP	3
CRYPTO_KE_KEYDERIVATION_ALGORITHM	CRYPTO_30_VHSM_ALGO_KDF_NIST_800_108_CMAC	8

Table 3-8 Custom Key Element Values

3.5.4 Custom Services

The communication with vHsm is done only with CRYPTO ASR4.3 compatible APIs. Custom behavior can be achieved by using custom keys and custom elements in combination with `KeyElementSet()` and `KeyElementGet()`. The vHsm catches this special keys and key element ids and performs the behavior described below instead of simply setting / reading a key. All other keys and elements are handled as usual.

3.5.4.1 Flash Access Synchronization

To synchronize flash access with other cores, vHsm provides a token mechanism. This mechanism allows an application core to restrict vHsm access to code flash and/or data flash.

Restricting vHsm data flash access will cause any request for the vHsm to write to flash to fail. For example, a call to `KeySetValid()` will fail when the vHsm doesn't has the flash token. Any other operation that does not access data flash at any point should still succeed.

Restricting vHsm code flash access will cause the vHsm to run in a ram loop. This will basically suspend the vHsm and the vHsm will not be able to process requests. Inside the

ram loop, the vHsm will call the callout function `vHsm_RamLoop_Callout()` where user can implement custom behavior.

Depending on the specific hardware it might not be necessary to use the flash token mechanism. It depends on which flash areas are accessed and how many flash controllers are used. For example, if the vHsm and an application core only use distinct flash controllers, it might not be necessary to further synchronize access via the token mechanism.

A common scenario is that the vHsm boots up, reads its key storage from flash, starts application cores and passes the data flash token to an application. Whenever the application wants the vHsm to persist something, it gives the token back to the vHsm, send the request and then request the token back after the vHsm is finished.

The following examples show the basic usage of the flash token. See 3.5.3.1 and 3.5.3.2 for the definitions of the key elements and key element values.



Example: Code flash access from application core

```
uint8 op[1];

/* Send vHsm to ram loop */
op[0] = CRYPTO_30_VHSM_CODEFLASH_START;
Csm_KeyElementSet(Hardware_Key, CRYPTO_KE_CUSTOM_VHSM_FLASH_OPERATION, op, 1);

/* Application can safely perform code flash operation here */

/* Release vHsm from ram loop */
op[0] = CRYPTO_30_VHSM_CODEFLASH_STOP;
Csm_KeyElementSet(Hardware_Key, CRYPTO_KE_CUSTOM_VHSM_FLASH_OPERATION, op, 1);
```



Example: Data flash access from application core

```
uint8 op[1];

/* Restrict data flash operations for vHsm */
op[0] = CRYPTO_30_VHSM_DATAFLASH_START;
Csm_KeyElementSet(Hardware_Key, CRYPTO_KE_CUSTOM_VHSM_FLASH_OPERATION, op, 1);

/* Application can safely perform data flash operation here */

/* Release vHsm from data flash restrictions */
op[0] = CRYPTO_30_VHSM_DATAFLASH_STOP;
Csm_KeyElementSet(Hardware_Key, CRYPTO_KE_CUSTOM_VHSM_FLASH_OPERATION, op, 1);
```

It is not important which key elements the Hardware_Key contains in the Configuration. It is only important that the key is routed to the Hal Crypto Driver which can be configured in /MICROSAR/vHsm/vHsmKeyMapping/vHsmKeyRouting.

3.6 True Random Number Generator

3.6.1 Prefetching of randomness

A ring buffer of size VHSM_HAL_TRNG_PREFETCH_CAPACITY[1] can be used to prefetch random numbers from the TRNG module. Prefetching can be activated by turning on VHSM_HAL_TRNG_PREFETCHING[2]. In addition, there are two modes of prefetching supported. Prefetching random data with interrupts and prefetching via polling random data. When VHSM_HAL_TRNG_PREFETCH_WITH_INTERRUPT[3] is turned on, interrupt mode is enabled, otherwise polling mode is enabled.

Using HSM_HAL_TRNG_PREFETCHING leads to more performant jobs that require a small amount of random data. When VHSM_HAL_TRNG_PREFETCH_WITH_INTERRUPT is enabled the buffer with random data is filled in a concurrent manner and will be more performant than the cyclic polling for new random data.

[1] /MICROSAR/vHsm_Hal/Crypto/CryptoGeneral/CryptoTrngPrefetchCapacity

[2] /MICROSAR/vHsm_Hal/Crypto/CryptoGeneral/CryptoTrngPrefetching

[3] /MICROSAR/vHsm_Hal/Crypto/CryptoGeneral/CryptoTrngPrefetchWithInterrupt

4 Integration

This chapter gives necessary information for the integration of the MICROSAR VHSM_HAL into an application environment of an ECU.

4.1 Scope of Delivery

The delivery of the VHSM_HAL contains the files which are described in the chapters 4.1.1 and 4.1.2.

There are additional files which contain to the platforms specific implementation of the component. These files are documented in vHsm_Hal.h header file.

4.1.1 Static Files

File Name	Description
Crypto_30_vHsm_Hal.h	This header file is for redirecting the `Crypto_30_vHsm_Hal_<Fn>` names to the vHsm_Hal component.
vHsm_Hal.c	This is the main source files of the vHsm_Hal. It contains functions for initialization of the component, the main function and job processing.
vHsm_Hal.h	This is the header file of the vHsm_Hal. It contains the revision history of the base component and the platform specific implementation.
vHsm_Hal_Aead.c	This is the source file for AEAD. It contains AEAD job processing functions.
vHsm_Hal_Cipher.c	This is the source file for Cipher primitives. It contains AES job processing functions.
vHsm_Hal_Mac.c	This is the source file for Mac primitives. It contains CMAC job processing functions.
vHsm_Hal_Custom.h	This header file defines custom crypto algorithm families supported by the vHsm_Hal.
vHsm_Hal_Hash.c	This is the source file for HASH primitives. It contains SHA256 job processing functions.
vHsm_Hal_Hw.h	This header files declares the general functions which are to be implemented by the platform specific implementation.
vHsm_Hal_KeyM.c	This source file contains Key Management functions.
vHsm_Hal_Random.c	This is the source file for RANDOM. It contains RANDOM job processing functions.
vHsm_Hal_Private.h	This header files contains the declaration of private functions intended to be used inside the vHsm_Hal component.
vHsm_Hal_Public.h	This is the main header file of the vHsm_Hal. It contains the public function declarations for the API provided by the component.
vHsm_Hal_Types.h	This header file defines the workspaces used by the crypto algorithms of the component.
vHsm_Hal_Hw_Cfg.h	This header file contains platform specific configuration elements.
vHsm_Hal_Hw_Types.h	This header file contains platform specific type definitions.

File Name	Description
vHsm_Hal_Hw.c *	This file contains the low-level implementation of general hardware access functions.
vHsm_Hal_Hw_Crypto.c *	This file contains the low-level implementation of cryptographic hardware access functions.
vHsm_Hal_Hw_Irq.c *	This contains the low-level implementation of interrupt service handlers.

Table 4-1 Static files

* The availability of this files depends on the platform specific implementation.

4.1.2 Dynamic Files

The dynamic files are generated by the configuration tool [config tool].

File Name	Description
vHsm_Hal_Cfg.c	This is generated source of the components configuration.
vHsm_Hal_Cfg.h	This is generated header of the components configuration.

Table 4-2 Generated files

4.2 Critical Sections

> VHSM_HAL_EXCLUSIVE_AREA_0

This critical section protects the vHsm_Hal crypto driver objects on lock and release. It also protects the locking and releasing of crypto keys.

> VHSM_HAL_EXCLUSIVE_AREA_1

This critical section protects access to Hardware for AES based crypto accelerators.

Protected primitives: AES, AEAD, CMAC

> VHSM_HAL_EXCLUSIVE_AREA_2

This critical section protects access to Hardware for SHA256 crypto accelerators.

> VHSM_HAL_EXCLUSIVE_AREA_3

This critical section protects access to Hardware for TRNG crypto accelerators.

4.3 Configuration

The vHsm_Hal configuration has a container called “CryptoPrimitives” which contains the preconfigured primitives which are supported by the platform and its implementation. Adding custom primitives in this container is a misconfiguration and leads to rejected crypto jobs.

The container “CryptoDriverObjects” can be customized with the crypto primitives which should be offered by the vHsm_Hal.

4.4 Integration Hints

There is an additional technical reference manual in the delivery with platform specific documentation. This includes the memory configuration for the IPC and the description of Interrupts provided by the vHsm_Hal.

See [6] vHsm_IntegrationHints_<Platform>.pdf for more information.

5 API Description

5.1 Services provided by vHsm_Hal

5.1.1 General services

5.1.1.1 vHsm_Hal_Init

Prototype	
`void vHsm_Hal_Init (void)`	
Parameter	
void	none
Return code	
void	none
Functional Description	
Initializes the vHsm Hardware Abstraction Library. This function initializes the module vHsm_Hal. It initializes all variables and sets the module state to initialized.	
Particularities and Limitations	
Module is uninitialized.	
Call context	
> TASK > This function is Synchronous > This function is Non-Reentrant	

Table 5-1 vHsm_Hal_Init

5.1.1.2 vHsm_Hal_InitMemory

Prototype	
`void vHsm_Hal_InitMemory (void)`	
Parameter	
void	none
Return code	
void	none
Functional Description	
Initializes variables and memory areas that cannot be initialized with the startup code. Initialize component variables at power up.	
Particularities and Limitations	
Module is uninitialized.	
Call context	

> TASK > This function is Synchronous > This function is Non-Reentrant

Table 5-2 vHsm_Hal_InitMemory

5.1.1.3 vHsm_Hal_MainFunction

Prototype	
`void vHsm_Hal_MainFunction (void)`	
Parameter	
void	none
Return code	
void	none
Functional Description	
Main function of the module. Is called cyclically.	
Particularities and Limitations	
> Declared and called by SchM > Is called cyclically.	
Call context	
> TASK > This function is Synchronous > This function is Non-Reentrant	

Table 5-3 vHsm_Hal_MainFunction

5.1.2 Crypto services

5.1.2.1 vHsm_Hal_ProcessJob

Prototype	
`Std_ReturnType` **`vHsm_Hal_ProcessJob`** `(uint32 objectId, Crypto_JobType *job)`	
Parameter	
objectId [in]	Holds the identifier of the Crypto Driver Object.
job [in,out]	Pointer to the configuration of the job. Contains structures with job and primitive relevant information but also pointer to result buffers.
Return code	
Std_ReturnType	E_OK Request successful.
	E_NOT_OK Request failed.
	CRYPTO_E_BUSY Request failed, Crypto Driver Object is busy.
	CRYPTO_E_KEY_NOT_VALID Request failed, the key is not valid.
	CRYPTO_E_QUEUE_FULL Request failed, the queue is full.
	CRYPTO_E_SMALL_BUFFER Request failed, the provided buffer is too small to store the result.
Functional Description	
Processes the received job. Performs the crypto primitive, that is configured in the job parameter.	
Particularities and Limitations	
-	
Call context	
> TASK > This function is Synchronous > This function is Reentrant	

Table 5-4 vHsm_Hal_ProcessJob

5.1.2.2 vHsm_Hal_CancelJob

Prototype	
`Std_ReturnType` **`vHsm_Hal_CancelJob`** `(uint32 objectId, Crypto_JobType *job)`	
Parameter	
objectId [in]	Holds the identifier of the Crypto Driver Object.
job [in,out]	Pointer to the configuration of the job. Contains structures with user and primitive relevant information.
Return code	
Std_ReturnType	E_OK Request successful, job has been removed.
Std_ReturnType	E_NOT_OK Request failed, job could not be removed.

Functional Description
Cancels the received job. This interface removes the provided job from the queue and cancels the processing of the job if possible.
Particularities and Limitations
-
Call context
> TASK > This function is Synchronous > This function is Reentrant

Table 5-5 vHsm_Hal_CancelJob

5.1.3 Key Management services

5.1.3.1 vHsm_Hal_KeyElementGet

Prototype	
`Std_ReturnType` **`vHsm_Hal_KeyElementGet`** `(uint32 cryptoKeyId, uint32 keyElementId, uint8 *resultPtr, uint32 *resultLengthPtr)`	
Parameter	
cryptoKeyId [in]	Holds the identifier of the key whose key element shall be set.
keyElementId [in]	Holds the identifier of the key element which shall be set.
resultPtr [out]	Holds the pointer to the key data which shall be set as key element.
resultLengthPtr [in,out]	Contains the length of the key element in bytes.
Return code	
Std_ReturnType	E_OK Request successful.
	E_NOT_OK Request failed.
	CRYPTO_E_BUSY Request failed, Crypto Driver Object is busy.
	CRYPTO_E_KEY_READ_FAIL Request failed, read access was denied.
	CRYPTO_E_KEY_NOT_AVAILABLE Request failed, the key is not available.
	CRYPTO_E_KEY_SIZE_MISMATCH Request failed, the provided buffer is too small to store the result.
Functional Description	
This interface shall be used to get a key element of the key identified by the cryptoKeyId and store the key element in the memory location pointed by the result pointer.	
Particularities and Limitations	
-	
Call context	
> TASK > This function is Synchronous > This function is Non-Reentrant	

Table 5-6 vHsm_Hal_KeyElementGet

5.1.3.2 vHsm_Hal_KeyElementSet

Prototype	
`Std_ReturnType vHsm_Hal_KeyElementSet (uint32 cryptoKeyId, uint32 keyElementId, const uint8 *keyPtr, uint32 keyLength)`	
Parameter	
cryptoKeyId [in]	Holds the identifier of the key whose key element shall be set.
keyElementId [in]	Holds the identifier of the key element which shall be set.
keyPtr [in]	Holds the pointer to the key data which shall be set as key element.
keyLength [in]	Contains the length of the key element in bytes.
Return code	
Std_ReturnType	E_OK Request successful.
	E_NOT_OK Request failed.
	CRYPTO_E_BUSY Request failed, Crypto Driver Object is busy.
	CRYPTO_E_KEY_WRITE_FAIL Request failed, write access was denied.
	CRYPTO_E_KEY_NOT_AVAILABLE Request failed, the key is not available.
	CRYPTO_E_KEY_SIZE_MISMATCH Request failed, the key element size does not match size of provided
	data.
Functional Description	
Sets a key element. Sets the given key element bytes to the key identified by cryptoKeyId. .	
Particularities and Limitations	
-	
Call context	
> TASK > This function is Synchronous > This function is Non-Reentrant	

Table 5-7 vHsm_Hal_KeyElementSet

5.1.3.3 vHsm_Hal_KeyDerive

Prototype	
`Std_ReturnType vHsm_Hal_KeyDerive (uint32 cryptoKeyId, uint32 targetCryptoKeyId)`	
Parameter	
cryptoKeyId [in]	Holds the identifier of the key which is used for key derivation
targetCryptoKeyId [in]	Holds the identifier of the key which is used to store the derived key.

Return code	
Std_ReturnType	E_OK Request successful
	E_NOT_OK Request Failed
	E_BUSY Request failed, Crypto Driver Object is busy
	CRYPTO_E_KEY_EMPTY Request failed because of uninitialized source key element.
Functional Description	
Derives a new key by using the key elements in the given key identified by the cryptoKeyId and stores the new key in key element (ID:1) of the targetCryptoKeyId.	
Particularities and Limitations	
Call context	
> TASK > This function is Synchronous > This function is Reentrant	

Table 5-8 vHsm_Hal_KeyDerive

5.1.3.4 vHsm_Hal_KeyValidSet

Prototype	
`Std_ReturnType vHsm_Hal_KeyValidSet (uint32 cryptoKeyId)`	
Parameter	
cryptoKeyId [in]	Holds the identifier of the key whose key elements shall be set to valid.
Return code	
Std_ReturnType	E_OK Request successful. E_NOT_OK Request failed. CRYPTO_E_BUSY Request failed, Crypto Driver Object is busy.
Functional Description	
Sets the key to valid. Sets the key state of the key identified by cryptoKeyId to valid.	
Particularities and Limitations	
-	
Call context	
> TASK > This function is Synchronous > This function is Reentrant	

Table 5-9 vHsm_Hal_KeyValidSet

5.1.4 Platform services

5.1.4.1 vHsm_Hal_GetDynamicMemoryAreaConfig

Prototype	
`void vHsm_Hal_GetDynamicMemoryAreaConfig (uint32 memoryAreaId, vHsm_Hal_MemoryAreaConfigType *vHsm_Hal_MemoryAreaConfig)`	
Parameter	
memoryAreaId [in]	Id of the memory area.
Pointer [in,out]	to memory area which can be modified by the vHsm_Hal
Return code	
void	none
Functional Description	
Function which enables the vHsm_Hal to modify a memory area during runtime.	
Particularities and Limitations	
-	
Call context	
> ANY > This function is Synchronous > This function is Reentrant	

Table 5-10 vHsm_Hal_GetDynamicMemoryAreaConfig

5.1.4.2 vHsm_Hal_SystemInit

Prototype	
`void vHsm_Hal_SystemInit (void)`	
Parameter	
void	none
Return code	
void	none
Functional Description	
Initialize the system hardware. This function can be called while vHsm_Core in initializing.	
Particularities and Limitations	
-	
Call context	
> ANY > This function is Synchronous > This function is Non-Reentrant	

Table 5-11 vHsm_Hal_SystemInit

5.1.4.3 vHsm_Hal_TriggerSoftwareReset

Prototype	
`void vHsm_Hal_TriggerSoftwareReset (void)`	
Parameter	
void	none
Return code	
void	none
Functional Description	
Trigger a software reset.	
Particularities and Limitations	
-	
Call context	
> ANY > This function is Synchronous > This function is Non-Reentrant	

Table 5-12 vHsm_Hal_TriggerSoftwareReset

5.1.4.4 vHsm_Hal_HandleResetInfo

Prototype	
`void vHsm_Hal_HandleResetInfo (void)`	
Parameter	
void	none
Return code	
void	none
Functional Description	
Handle the reset Info and all actions which are related to it like changing the reset vectors. After a software reset, the reset info may contain instruction which have to be performed like changing reset vectors.	
Particularities and Limitations	
-	
Call context	
> TASK > This function is Synchronous > This function is Non-Reentrant	

Table 5-13 vHsm_Hal_HandleResetInfo

5.1.4.5 vHsm_Hal_StartupCallout

Prototype	
`void vHsm_Hal_StartupCallout (void)`	
Parameter	
void	none
Return code	
void	none
Functional Description	
Callout which is called during startup at the very beginning. This function is called from the startup code. This is for special hardware initialization at the very beginning.	
Particularities and Limitations	
-	
Call context	
> ANY > This function is Synchronous > This function is Non-Reentrant	

Table 5-14 vHsm_Hal_StartupCallout

5.1.4.6 vHsm_Hal_TriggerInternalIrq

Prototype	
`void vHsm_Hal_TriggerInternalIrq (void)`	
Parameter	
void	none
Return code	
void	none
Functional Description	
Trigger internal Cat2 interrupt. -	
Particularities and Limitations	
-	
Call context	
> ANY > This function is Synchronous > This function is Reentrant	

Table 5-15 vHsm_Hal_TriggerInternalIrq

5.1.4.7 vHsm_Hal_ApplicationIpcInit

Prototype	
`boolean` **`vHsm_Hal_ApplicationIpcInit`** `(vHsm_IpcInstanceIterType ipc)`	
Parameter	
ipc [in]	IPC iterator
Return code	
boolean	TRUE IPC instance has been initialized by application core FALSE IPC instance has not been initialized by application core
Functional Description	
Checks if the IPC instance given by ipcIdx is initialized by application core. -	
Particularities and Limitations	
-	
Call context	
> TASK > This function is Synchronous > This function is Reentrant	

Table 5-16 vHsm_Hal_ApplicationIpcInit

5.1.4.8 vHsm_Hal_EnableInterruptSource

Prototype	
`void` **`vHsm_Hal_EnableInterruptSource`** `(uint32 sources)`	
Parameter	
sources [in]	Bitmask to only enable interrupts for specific host cores
Return code	
void	none
Functional Description	
Enables job request interrupts from host cores. -	
Particularities and Limitations	
-	
Call context	
> TASK > This function is Synchronous > This function is Reentrant	

Table 5-17 vHsm_Hal_EnableInterruptSource

5.1.4.9 vHsm_Hal_DisableInterruptSource

Prototype	
`void vHsm_Hal_DisableInterruptSource (uint32 sources)`	
Parameter	
sources [in]	Bitmask to only disable interrupts for specific host cores
Return code	
void	none
Functional Description	
Disables job request interrupts from host cores. -	
Particularities and Limitations	
-	
Call context	
> TASK > This function is Synchronous > This function is Reentrant	

Table 5-18 vHsm_Hal_DisableInterruptSource

5.1.5 Optional callouts

The callouts described in this chapter can be used by the HSM system to for specific hardware dependent behavior. These callouts are optional and only implemented by the platform implementation if needed.

5.1.5.1 vHsm_Hal_GetCodeFlashAddr

Prototype	
`const uint8 * vHsm_Hal_GetCodeFlashAddr (const uint8 *offset)`	
Parameter	
offset [in]	Pointer wich represents the current code flash offset.
Return code	
const uint8 *	Pointer of the code flash offset.
Functional Description	
Get the address of the code flash. Used for the VTT simulation to redirect the code flash address to the memory mapped file.	
Particularities and Limitations	
-	
Call context	
> ANY	

> This function is Synchronous > This function is Reentrant

Table 5-19 vHsm_Hal_GetCodeFlashAddr

5.1.5.2 vHsm_Hal_PrepareCryptoJobData

Prototype	
`void vHsm_Hal_PrepareCryptoJobData (Crypto_JobPrimitiveInputOutputType *jobInputOutput)`	
Parameter	
jobInputOutput [in,out]	The jobs primitive input output data.
Return code	
void	none
Functional Description	
Prepare input and output pointers in Crypto job for access from HSM side.	
Particularities and Limitations	
-	
Call context	
> TASK > This function is Synchronous > This function is Non-Reentrant	

Table 5-20 vHsm_Hal_PrepareCryptoJobData

5.1.5.3 vHsm_Hal_PrepareKeyMJobData

Prototype	
`void vHsm_Hal_PrepareKeyMJobData (vHsm_KeyM_InputOutputType *jobInputOutput)`	
Parameter	
jobInputOutput [in,out]	The jobs primitive input output data.
Return code	
void	none
Functional Description	
Prepare input and output pointers in KeyM job for access from HSM side.	
Particularities and Limitations	
-	
Call context	
> TASK > This function is Synchronous	

> This function is Non-Reentrant

Table 5-21 vHsm_Hal_PrepareKeyMJobData

5.1.5.4 vHsm_Hal_GetIpcAddress

Prototype	
`const uint32 * vHsm_Hal_GetIpcAddress (uint32 index, const uint32 *address)`	
Parameter	
index [in]	Index of the current IPC requested by the vHsm_Core
address [in]	Memory address of the current IPC requested by the vHsm_Core
Return code	
const uint32 *	A pointer to the memory which shall be used for the chosen IPC index.
Functional Description	
Get the address if the IPC. This callout can be used by HAL to provide own IPC memory address or to rewrite the given address.	
Particularities and Limitations	
-	
Call context	
> TASK > This function is Synchronous > This function is Non-Reentrant	

Table 5-22 vHsm_Hal_GetIpcAddress

5.1.5.5 vHsm_Hal_EnableCodeFlashAccess

Prototype	
`void vHsm_Hal_EnableCodeFlashAccess (void)`	
Parameter	
void	none
Return code	
void	none
Functional Description	
Allow the write access to the code flash by the application core.	
Particularities and Limitations	
-	
Call context	
> ANY	

> This function is Synchronous > This function is Non-Reentrant

Table 5-23 vHsm_Hal_EnableCodeFlashAccess

5.1.5.6 vHsm_Hal_DisableCodeFlashAccess

Prototype	
`void vHsm_Hal_DisableCodeFlashAccess (void)`	
Parameter	
void	none
Return code	
void	none
Functional Description	
Prevent the write access to the code flash by the application core.	
Particularities and Limitations	
-	
Call context	
> ANY > This function is Synchronous > This function is Non-Reentrant	

Table 5-24 vHsm_Hal_DisableCodeFlashAccess

5.1.5.7 vHsm_Hal_IsDebuggerPresent

Prototype	
`void vHsm_Hal_IsDebuggerPresent (void)`	
Parameter	
void	none
Return code	
boolean	TRUE Debugger is present FALSE Debugger is not present
Functional Description	
Checks whether a debugger is connected to the chip.	
Particularities and Limitations	
-	
Call context	
> TASK > This function is Synchronous	

>	This function is Reentrant

Table 5-25 vHsm_Hal_IsDebuggerPresent

5.1.6 Interrupt Service Routines provided by VHSM_HAL

VHSM_HAL provides two interrupt service routines to handle job requests from an application core. `vHsm_Hal_ExternalRequestIsr()` is used to receive a job request from the application core itself. Depending on the category of the interrupt, there might be an additional prefix for this function. Category 1 interrupts are not handled from an Os and category 2 interrupts are handled by an Os. Depending on the specific hardware platform, `vHsm_Hal_InternalRequestIsr()` is also needed to translate category 1 interrupts to category 2 interrupts when category 2 interrupts are not supported because interrupts may get lost. This function is triggered internally if needed.



Note
Select the request interrupt category in
/MICROSAR/vHsm_Hal/Crypto/CryptoGeneral/CryptoRequestIsrCategory

In case that TRNG prefetching is enabled, `vHsm_Hal_Hw_TrngIRQHandler()` is used to handle interrupts from the TRNG module when new random data was generated. This interrupt has to be always a category 1 interrupt.



Note
TRNG prefetching can be enabled in
/MICROSAR/vHsm_Hal/Crypto/CryptoGeneral/CryptoTrngPrefetching

See [6] vHsm_IntegrationHints_<Platform>.pdf for platform specific information.

5.1.7 Services used by VHSM_HAL

The following table lists services which are provided by other modules and used by the VHSM_HAL. For details about prototypes and functionality refer to the documentation of the providing component.

Component	API
DET	Det_ReportError()
VHSM (vHsm_Core)	vHsm_Api_KeyReadLockGetNotProtected() vHsm_Api_KeyReadLockReleaseNotProtected() vHsm_Api_GetAddrKeyStorage() vHsm_Api_KeyElementGetForCryptoJobs() vHsm_Api_KeyValidSet() vHsm_Api_ProcessTempJob() vHsm_Api_InitializeTempJob()

Component	API
	vHsm_Api_TempJob_SetPrimitive() vHsm_Api_KeyElementSet() vHsm_Api_KeyElementGet() vHsm_Api_Local_KeyElementGetStorageIndex() vHsm_Api_Local_KeyElementSearch() vHsm_Api_GetKeyElementLength() vHsm_Api_KeyReadLockGetNotProtected() vHsm_Api_KeyReadLockReleaseNotProtected() vHsm_ExternalIsr() vHsm_InternalIsr()
VSTDLIB	VStdLib_MemClr() VStdLib_MemCpy() VStdLib_MemSet()

Table 5-26 Services used by the VHSM_HAL

6 Configuration

In the VHSM_HAL the attributes can be configured according to/ with the following methods/ tools:

- Configuration in DaVinci Configurator 5 Pro

6.1 Configuration Variants

The VHSM_HAL supports the configuration variants

- `VARIANT-PRE-COMPILE`

The configuration classes of the VHSM_HAL parameters depend on the supported configuration variants. For their definitions please see the vHsm_Hal_bswmd.arxml or vHsm_Hal_<Platform>_bswmd.arxml file.

7 Glossary and Abbreviations

7.1 Glossary

Term	Description
CRYPTO	Crypto driver

Table 7-1 Glossary

7.2 Abbreviations

Abbreviation	Description
AAD	Additional Authenticated Data
AEAD	Authenticated Encryption with Associated Data
AES	Advanced Encryption Standard
IPC	Inter Processor Communication
HAL	Hardware Abstraction Layer
ALGOFAM	Algorithm Family
ALGOMODE	Algorithm Mode
CBC	Cipher Block Chaining Mode
CMAC	Cipher-based Message Authentication Code
CTR	Counter Mode
ECB	Electronic Code Book
FIPS	Federal Information Processing Standards Publication
GCM	Galois Counter Mode
NIST	National Institute of Standards and Technology
RNG	Random Number Generator
SHA	Secure Hash Algorithm
SHE	Secure Hardware Extension
TRNG	True Random Number Generator
API	Application Programming Interface
AUTOSAR	Automotive Open System Architecture
BSW	Basis Software
DET	Development Error Tracer
ECU	Electronic Control Unit
ISR	Interrupt Service Routine
MICROSAR	Microcontroller Open System Architecture (the Vector AUTOSAR solution)
SRS	Software Requirement Specification
SWC	Software Component
SWS	Software Specification

Table 7-2 Abbreviations

8 Contact

Visit our website for more information on

- News
- Products
- Demo software
- Support
- Training data
- Addresses

www.vector.com